Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

CollabCode : Personal AI developer with chat application

Student/Team Information

Team Name: Team # (Mentor needs to assign)	DevFusion
Team member 1 (Team Lead)	Raj, Prince : 230112331 princeraj.connect@gmail.com

Team member 2	Mittal, Raghav : 230213884 raghavmittal518@gmail.com

Team member 3	Verma, Yatharth - 230211178 Vermayath2023@gmail.com
Team member 4	Anand, Nikhil : 230213860 nikhilanand0904@gmail.com

PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract

CollabCode AI is basically a shared space where teams can chat and build software at the same time. Instead of switching between tools, everything happens in one place. It's built on the MERN stack and uses chat rooms where developers can jump in, discuss ideas, and work on projects together as they go.

A big part of what makes it useful is the AI Developer Agent, powered by Google's Gemini API. It doesn't just throw out code suggestions—it can put together entire project setups. If you tag @AI in the chat, it can generate something like a ready-to-use Express server based on your request.

Another nice touch is how the generated code is handled. You don't have to copy anything manually. The AI creates a full file structure, and it's loaded straight into a Node.js environment in your browser using the WebContainers API. From there, you can open it, edit it, and run it right away without installing anything.

At the end of the day, it's really about saving time—especially on the repetitive setup stuff—so teams can focus more on building and trying out ideas.

Updated Project Approach and Architecture

The system follows a typical client–server approach, with WebSockets taking care of most of the real-time interactions.

On the frontend, React and Tailwind CSS help keep the interface quick and responsive. API calls are handled with axios, and socket.io-client manages live communication. For the in-browser coding setup, WebContainers allows us to load file structures and run Node.js directly in the browser, so there's no need for any local environment.

On the backend, Node.js and Express handle routing and core logic. MongoDB, used with Mongoose, stores everything from user data to project details and saved file trees.

Authentication is based on JWTs stored in HTTP-only cookies, which adds an extra layer of security. Redis is used alongside this to make token verification faster and to handle cases like logging users out by invalidating tokens.

Real-time collaboration is managed through Socket.io, where each project gets its own isolated room. This keeps messages and AI-generated responses limited to the right group of users.

For the AI side of things, the Google Gemini API is guided with structured prompts so it consistently returns JSON-based file trees instead of plain text. That makes it straightforward to plug the output directly into the system and start using it right away.

Tasks Completed

Task Completed	Team Member
Authentication System: Built JWT-based login/register flow with bcrypt password hashing and Redis token blacklisting.	Prince Raj
Project & Room Management: Developed backend schemas to link users to specific projects and created the UI side-panel to manage active collaborators.	Nikhil Anand
Real-time Chat Engine**:** Integrated Socket.io for instant messaging within isolated project rooms.	Raghav Mittal
Gemini AI Integration: Engineered system prompts forcing the AI to return structured file trees based on chat requests.	Yatharth Verma

Challenges/Roadblocks

One of the trickiest issues we ran into was related to browser security when working with WebContainers. Since they need a properly isolated environment to run Node.js inside the browser, we kept hitting errors caused by missing COOP and COEP headers. It took a bit of digging, but we eventually traced it back to our Vite setup and updated the configuration to explicitly enable the required cross-origin isolation. Once that was in place, the containers started working as expected.

Another challenge was getting consistent responses from the Gemini API. Early on, the AI would mix normal text with markdown and code blocks, which made it hard for our frontend to parse the output reliably. To fix this, we spent time refining how we structured our prompts. We introduced stricter system instructions and included predefined JSON examples so the AI would stick to a predictable format. That way, the response always came back as a clean, structured object that our React app could easily process and turn into a working file tree.

Tasks Pending (7 pts)

Task Pending	Team Member (to complete the task)
Simulate multiple concurrent users in different Socket.io rooms to ensure the server handles the load without dropping connections.	Raghav Mittal

Project Outcome/Deliverables (2 pts)

In the end, we built a fully working browser-based collaborative workspace. The platform includes a secure authentication system, a real-time chat interface with project-specific rooms, and an integrated AI developer agent.

One of the main highlights is the interactive code execution panel. It lets users view the exact file structure generated by the AI, make changes to the code on the spot, and instantly see the output rendered within an embedded iframe. This makes it easy to go from generated code to a working application without leaving the platform.

Progress Overview (2 pts)

The project is about 95% complete and, overall, we're slightly ahead of schedule when it comes to core features. The database, backend APIs, WebSocket setup, AI prompting logic, and WebContainer integration are all working well in the local environment.

The only area where we're a bit behind is deployment. We're taking extra time to make sure things like CORS policies and WebContainer-related security headers behave correctly when moving from localhost to a live domain.

At this point, most of the heavy lifting is done. What's left is mainly deployment, some performance checks, and a bit of UI/UX polishing.

Codebase Information

Repository link : https://github.com/Nikhil4955/OWN_AMLI
Branch : main

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Unit Testing	Pass	Verified JWT generation, password hashing, and Redis blacklisting on logout.
WebSocket Connectivity	Pass	Confirmed messages are strictly isolated to specific project rooms.
AI JSON Parsing	Pass	Verified the frontend successfully catches and parses the AI's file-tree JSON schema without crashing.

Deliverables Progress

Backend & Socket Initialization : Completed
User Authentication (JWT/MongoDB/Redis) : Completed
React Frontend UI Development : Completed
Client-Side Socket Integration (UI Chat): Completed
Google Gemini API Integration : Completed
Real-time Collaboration Engine : Completed.